

SOA

from the Enterprise to the Cloud

February 2010

Version 1.0

Chris Swan
CTO

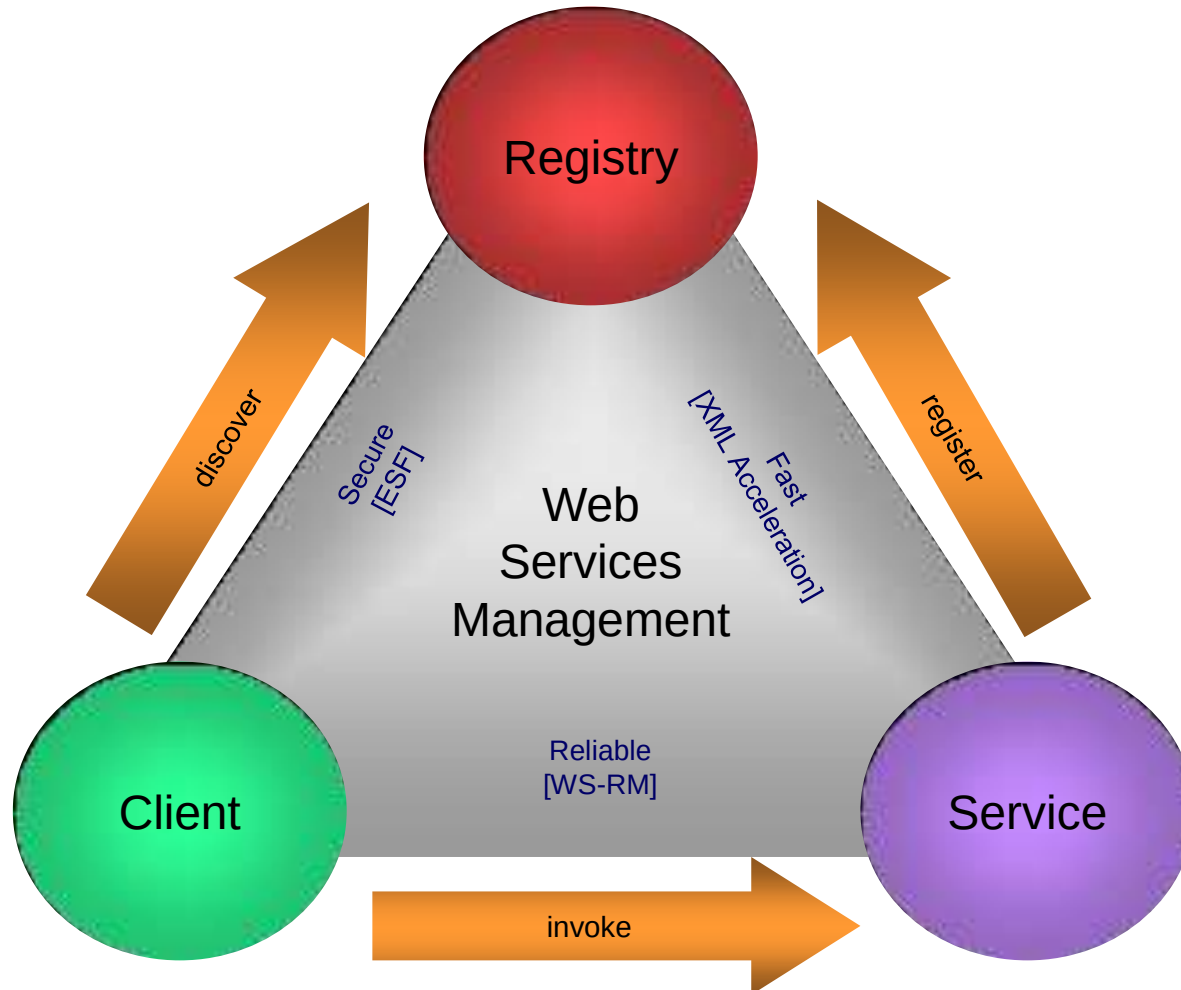
SOA – a personal journey that lasted 5 years

Take a trip back to 2004



- SOA wasn't new, but the emergence of Web Services gave it new life
- WS-* was still a smallish subset of 'standards'
 - SOAP
 - WSDL
 - UDDI
- Applications could now loosely couple across boundaries that had been hard to bridge before
 - Platform – Java or .Net
 - Language – managed code (more Java or .Net), unmanaged code (C++ etc.)
- The web had already solved plumbing problems at internet scale and in the enterprise
 - This would piggyback on that success
- Infrastructure would expose lots of management services
 - This would result in LOTS of endpoint to manage in a large enterprise

and this was what we ended up with...



But it all went horribly wrong



<http://www.loudthinking.com/arc/000585.html>

and we ended up with standards
about standards



Reference Documents	Profile Name	Basic 1.0	Basic 1.1	Basic 1.2	Basic 2.0	Attachments 1.0	Simple Soap Binding 1.0	Basic Security 1.0	Basic Security 1.1	Reliable Secure 1.0	Kerberos Token 1.0	REL Token Profile 1.0	SAML Token 1.0
Reference, Section 8				X	X								
UDDI Version 2.04 API Specification, Dated 19 July 2002		X	X	X	X								
WS-Addressing 1.0 - Core				X	X								
WS-Addressing 1.0 - Metadata (except for sections 4.1.1, 4.4.2, 4.4.3 and 5.2)				X	X								
WS-Addressing 1.0 - SOAP Binding										X			
WS-Addressing 1.0 - SOAP Binding										X			
WS-Addressing 1.0 - SOAP Binding (except for sections 2, 3, 5.1.2, 5.2.2 and 6.1)				X									

Too many things trying to make up
for the inadequacies of HTTP



WS-Federation

WS-Addressing

WS-Security

WS-RM

WS-Eventing

WS-Transfer

WS-MetadataExchange

WS-Discovery

WS-Policy

When what was really needed was a better (or cheaper) MOM

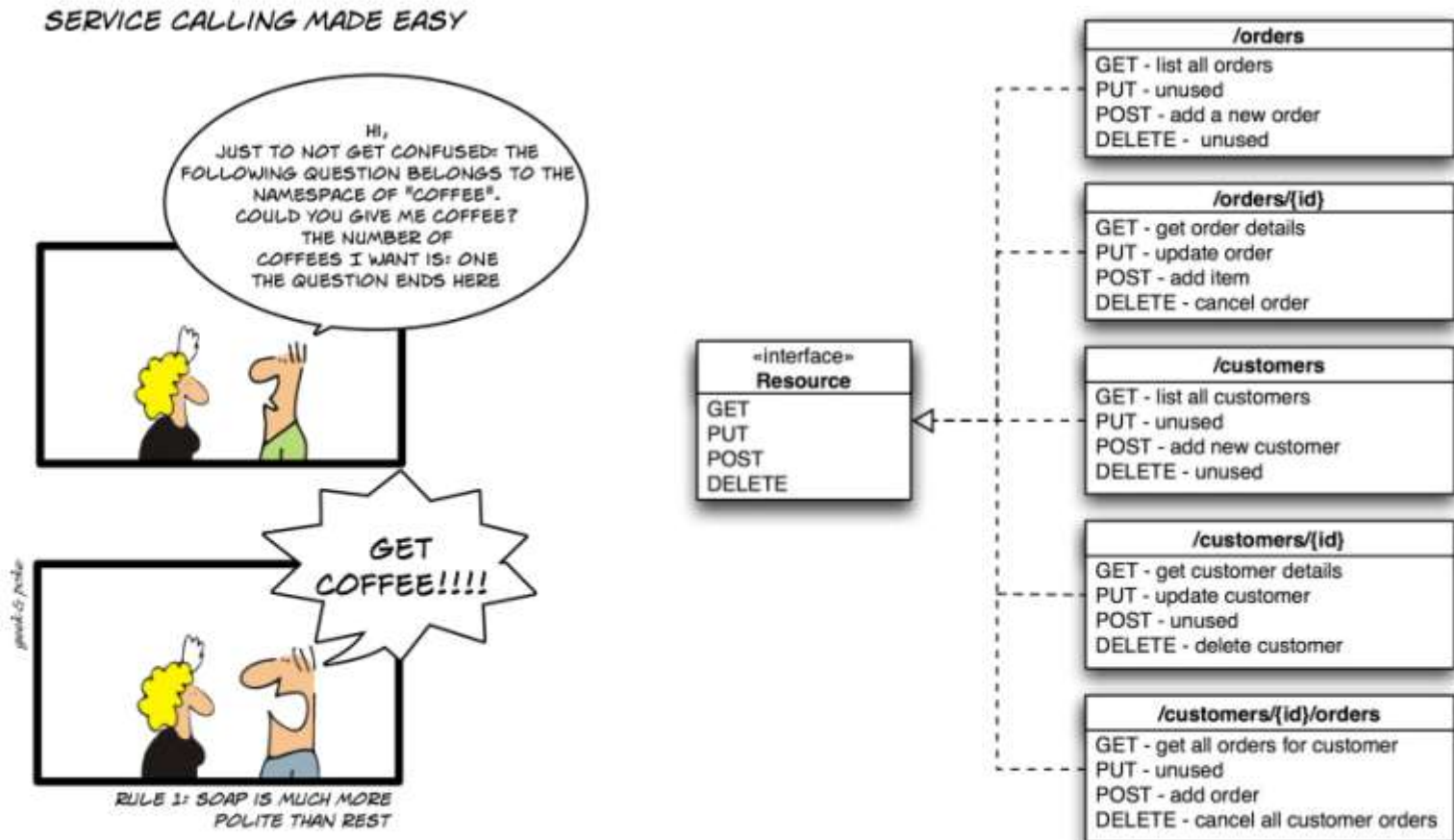


This is the
'magic quadrant'
of SOA

Async	SMTP	AMQP
Sync	HTTP	IIOP
	Unreliable	Reliable

But until recently
only proprietary
implementations

So the people who could live with HTTP retreated to simplicity



SOA doesn't fail because of techie stuff



SOA fails because of Conways Law



“...organizations which design systems
... are constrained to produce designs which
are copies of the communication structures of
these organizations”

Or... your org chart had better look like
your architecture, or you're doomed.

SOA succeeds when you have



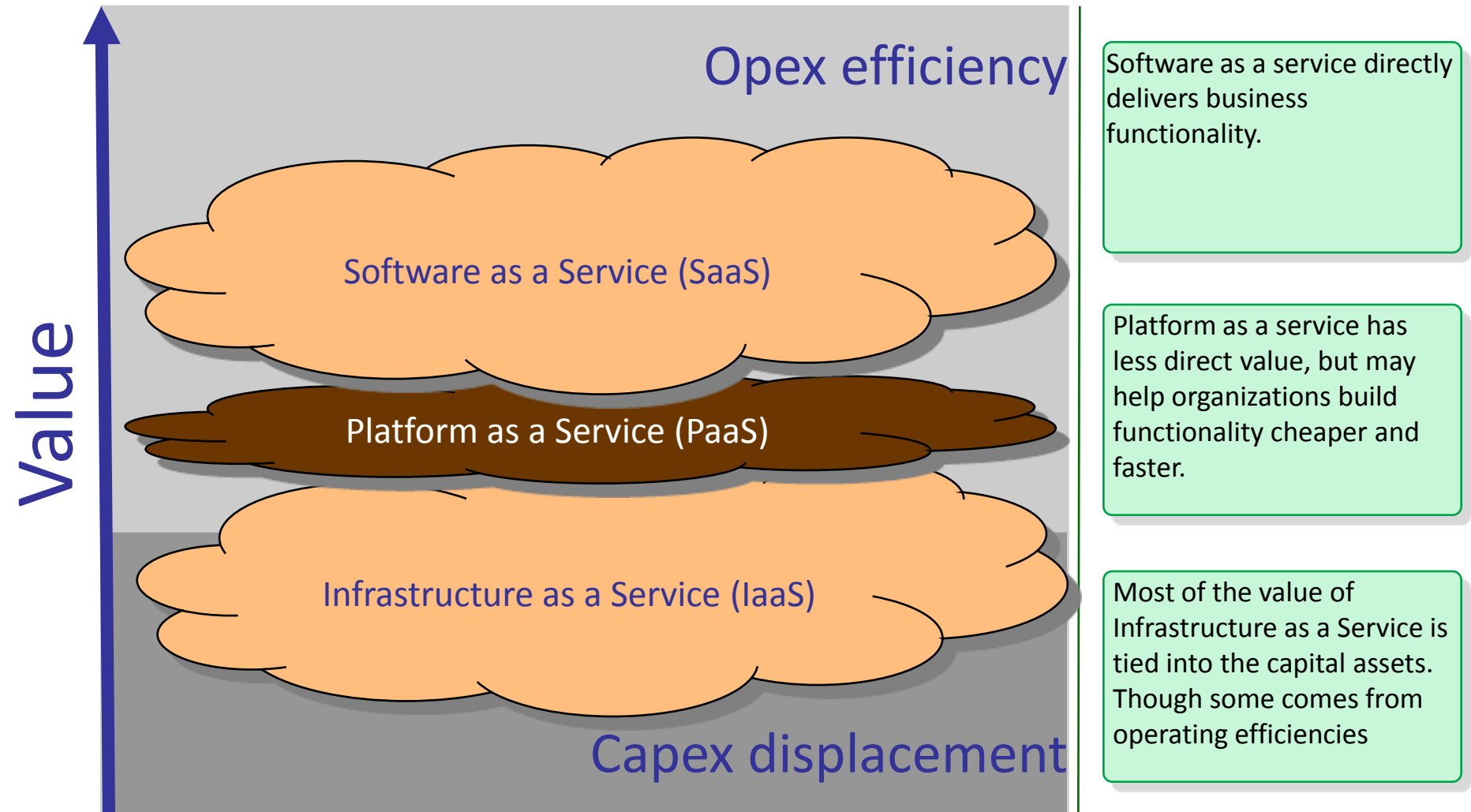
Command
and

Control



An then everything as a service came along

The Cloudburger



Back to the future – The EGA Technical Reference Model



Business process / service	Reference Data	Risk Management	Customer Portal
Virtualized Platform	Data Grid	Compute Grid	Server Farm
Platform Instance	Database	App Server	Web Server
Virtualized Operating Environment	NFS, SMB, NAS	Virtual Machine Monitors	Load balancing, VIPs
Operating Environment	File systems e.g. NTFS, Ext3	Operating Systems e.g. Linux, Windows	Network protocols e.g. TCP/IP, UDP
Virtualized Physical	LUNs	Hypervisors	VLANs
Physical	Disks, Array Controller, SAN switches etc. Storage	Servers, Blades etc. Compute	Switches, Routers etc. Network

Each physical layer provides Abstraction to the layer above

Each Virtualized layer provides a flexible mapping/management point

Where did the complexity go?



You chose to live without it

- Vanilla is good enough
- SaaS – everybody gets the same
 - Incremental upgrades
 - ‘Labs’ style features for the brave

or it gets squeezed out of one line of business (budget) and into another

- IaaS – S, M, L, XL
 - Complexities get pushed over the wall to application support
 - Ops -> DevOps
 - How much of this can be absorbed into PaaS layer?

The difference between 'hosting' and 'cloud'



It's all about the API

- But API is maybe the wrong term

But it is all about the Interface

- Invariably some kind of (HTTP) web service
 - Usually REST has won out over SOAP

